

SEMINAR

XÂY DỰNG WEBSITE SỬ DỤNG GITHUB COPILOT

Nguyễn Hồng Thắm

Lương Thị Thùy Quyên

Nguyễn Ngọc Nhiên

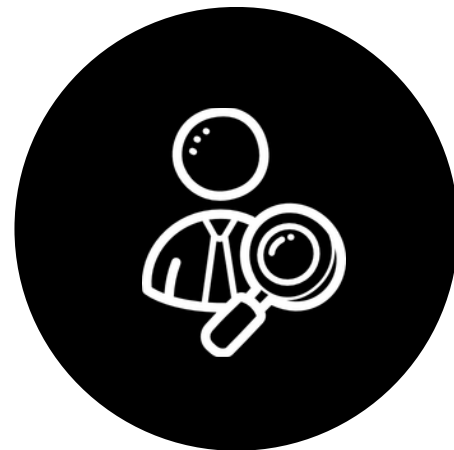
BỐI CẢNH BÀI TOÁN HIỆN NAY

ĐỘ PHỨC TẠP TĂNG CAO
CÁC HỆ THỐNG HIỆN ĐẠI YÊU CẦU THAY
ĐỔI CÔNG NGHỆ THƯỜNG XUYÊN

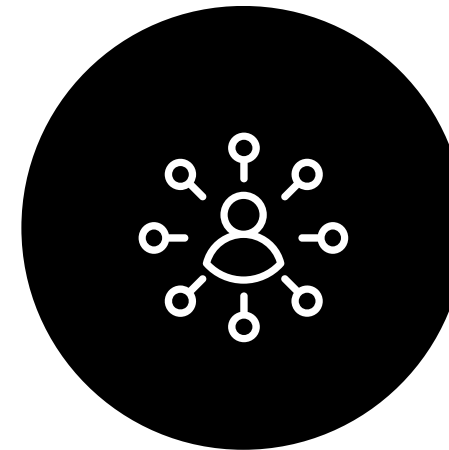
HẠN CHẾ NGUỒN LỰC
PHƯƠNG PHÁP TRUYỀN THỐNG GẶP RÀO
CẢN VỀ THỜI GIAN VÀ CHI PHÍ BẢO TRÌ

KỶ NGUYÊN AI
SỰ XUẤT HIỆN CỦA GITHUB COPILOT MỞ
RA KHẢ NĂNG "VIBE CODING" ĐỘT PHÁ

MỤC TIÊU VÀ PHẠM VI DỰ ÁN



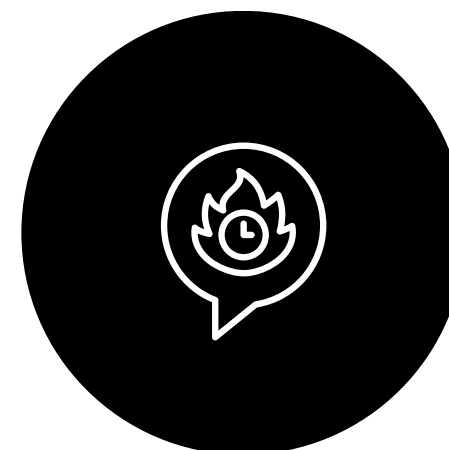
NGHIÊN CỨU AI-ASSISTED
DEVELOPMENT



XÂY DỰNG & DI TRÚ HỆ
THỐNG



ĐÁNH GIÁ PHƯƠNG PHÁP
LUẬN



PHẠM VI GIỚI HẠN

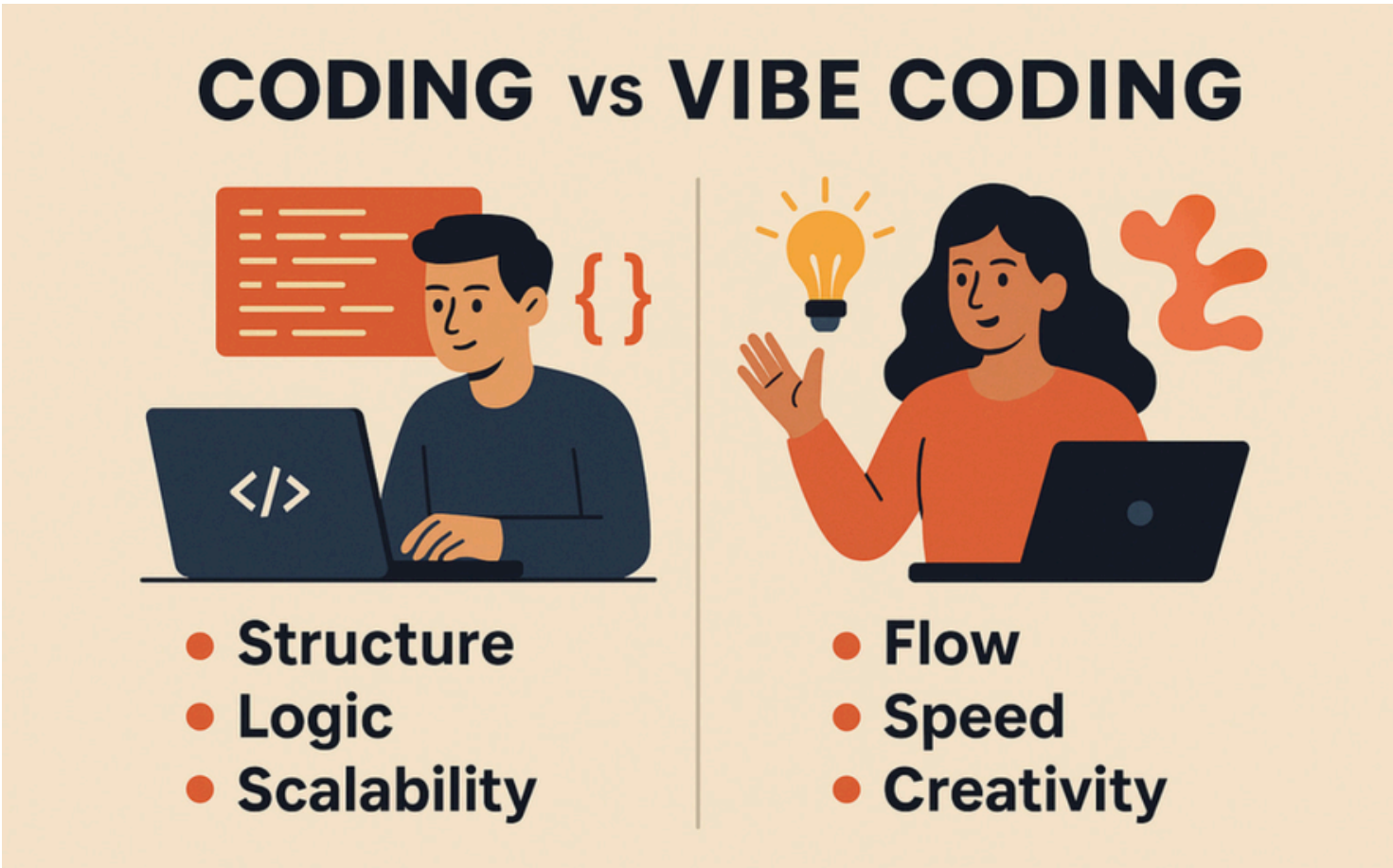
PHƯƠNG PHÁP TIẾP CẬN

Ưu tiên ý định (Intent) hơn cú pháp (Syntax).

Vibe Coding là một phong cách lập trình hiện đại, nơi lập trình viên không cần viết chi tiết từng dòng mã nguồn. Thay vào đó, họ diễn đạt ý tưởng, mục tiêu và yêu cầu bằng ngôn ngữ tự nhiên.

Các công cụ AI như GitHub Copilot Agent Mode hoạt động như một cộng tác viên kỹ thuật, có khả năng phân tích yêu cầu, tự động sinh mã nguồn hoàn chỉnh và thực hiện thay đổi trên nhiều tệp trong dự án.

CODING TRUYỀN THỐNG VỚI VIBE CODING



Tiêu chí	Coding truyền thống	Vibe Coding
Vai trò con người	Viết mã chi tiết	Định hướng, kiểm soát
Vai trò công cụ	IDE, thư viện	AI Agent (Copilot)
Cách tiếp cận	Code-first	Prompt-first
Thời gian phát triển	Dài, phụ thuộc kinh nghiệm	Nhanh hơn, linh hoạt
Khả năng di trú công nghệ	Khó, tốn công	Dễ hơn nhờ AI hỗ trợ

VAI TRÒ CỦA GITHUB COPILOT TRONG DỰ ÁN

Tác nhân hỗ trợ (Agent Mode)

GitHub Copilot không chỉ dừng ở việc gợi ý mã nguồn mà còn có khả năng hiểu ngữ cảnh toàn bộ dự án. Công cụ này có thể đề xuất cấu trúc hệ thống, tạo hoặc chỉnh sửa nhiều tệp tin đồng thời, đồng thời hỗ trợ thực hiện các thay đổi phức tạp trong quy trình phát triển phần mềm.

Hỗ trợ di trú công nghệ (Technology Migration)

Copilot hỗ trợ quá trình chuyển đổi mã nguồn giữa các công nghệ và nền tảng khác nhau. Thông qua việc phân tích logic nghiệp vụ của hệ thống hiện tại, công cụ có thể hỗ trợ tái cấu trúc và chuyển đổi sang kiến trúc hoặc ngôn ngữ mới, chẳng hạn từ Python sang Java Spring Boot hoặc từ React sang Blazor.

Tự động hóa các tác vụ lặp lại

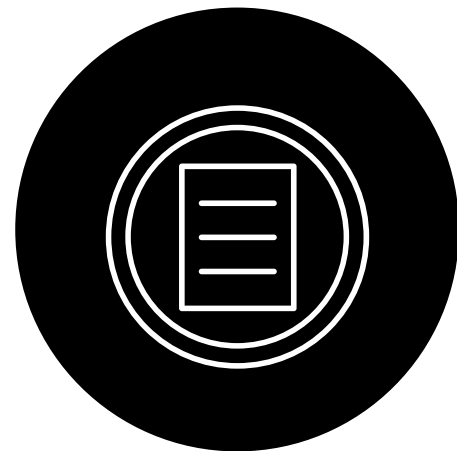
Copilot giúp giảm tải các công việc mang tính lặp lại bằng cách tự động sinh mã boilerplate, tạo tệp cấu hình như Dockerfile, đồng thời hỗ trợ xây dựng dữ liệu mẫu phục vụ kiểm thử và phát triển ứng dụng.

Bản này hợp kiểu báo cáo / slide / seminar hơn, đọc chuyên môn mà không quá “quảng cáo AI”.

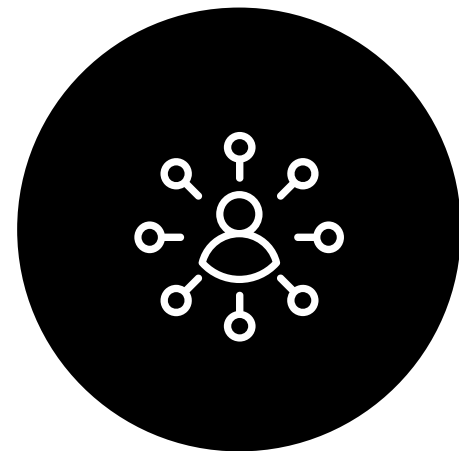
PROMPT DRIVEN DEVELOPMENT

PROMPT DRIVEN DEVELOPMENT LÀ KỸ THUẬT PHÁT TRIỂN PHẦN MỀM DỰA TRÊN VIỆC THIẾT KẾ PROMPT RÕ RÀNG, CÓ CẤU TRÚC ĐỂ ĐIỀU KHIỂN AI SINH RA KẾT QUẢ MONG MUỐN.

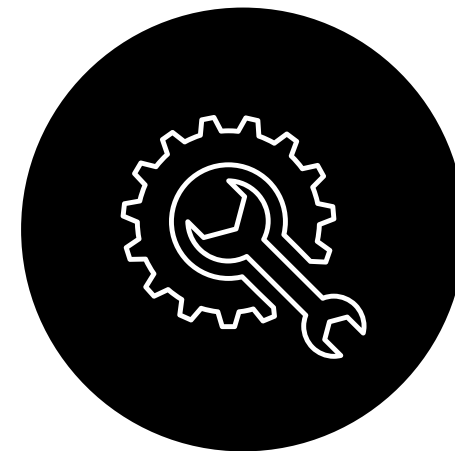
MỘT PROMPT HIỆU QUẢ THƯỜNG BAO GỒM:



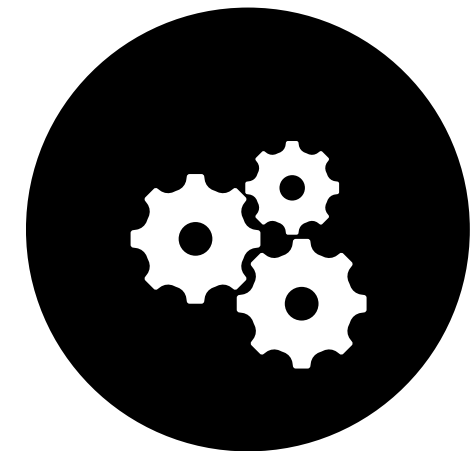
MÔ TẢ BỐI CẢNH
(CONTEXT)



YÊU CẦU CHỨC NĂNG
(WHAT)



RÀNG BUỘC KỸ THUẬT
(CONSTRAINTS)



CÔNG NGHỆ SỬ DỤNG
(TECH STACK)

PHÂN TÍCH HỆ THỐNG

COPILOT TRONG PHÂN TÍCH NGHIỆP VỤ

- Tóm tắt yêu cầu chính
- Copilot hỗ trợ phân tích và tổng hợp các mục tiêu cốt lõi của doanh nghiệp. Ví dụ, đối với công ty Contoso, hệ thống có thể xác định mục tiêu xây dựng một nền tảng mạng xã hội quy mô nhỏ nhằm quảng bá các sản phẩm liên quan đến lĩnh vực ngoài trời.
- Xác định các thực thể nghiệp vụ
- AI hỗ trợ nhận diện và liệt kê các đối tượng trọng tâm cần quản lý trong hệ thống, bao gồm: Người dùng (User), Bài viết (Post), Bình luận (Comment) cùng các thành phần liên quan phục vụ thiết kế ứng dụng.
- Hoặc bản ngắn gọn hơn kiểu slide thuyết trình:
- Sử dụng Copilot để xử lý file product-requirements.md
- Tóm tắt yêu cầu: Copilot phân tích và trích xuất các mục tiêu chính của dự án từ tài liệu yêu cầu.
- Xác định thực thể nghiệp vụ: Tự động nhận diện các đối tượng cốt lõi như User, Post, Comment phục vụ thiết kế hệ thống.
- Hỗ trợ hiểu ngữ cảnh dự án: Giúp chuyển đổi tài liệu yêu cầu thành cấu trúc dữ liệu và logic phát triển phù hợp.

PHÂN TÍCH HỆ THỐNG

COPILOT TRONG THIẾT KẾ HỆ THỐNG

Giai đoạn 1: Khởi tạo hệ thống

- **Mục tiêu :** Phát triển nhanh một phiên bản khả dụng tối thiểu (MVP – Minimum Viable Product) cho nền tảng Micro Social Media.
- **Công nghệ sử dụng:** Ứng dụng được xây dựng trên nền tảng Python (FastAPI) cho backend và JavaScript (React) cho frontend.
- **Vai trò của Copilot:** Copilot hỗ trợ tăng tốc quá trình phát triển thông qua việc sinh mã nguồn, đề xuất cấu trúc dự án và hỗ trợ xây dựng các chức năng cốt lõi của hệ thống.

PHÂN TÍCH HỆ THỐNG

Giai đoạn 2: Di trú công nghệ

Mục tiêu

Đánh giá khả năng của GitHub Copilot Agent Mode trong việc hỗ trợ chuyển đổi hệ thống.

Công nghệ

Backend: FastAPI → Java Spring Boot

Frontend: React → .NET Blazor

Vai trò Copilot

Phân tích mã nguồn cũ, đề xuất cấu trúc mới và tự động sinh mã chuyển đổi.

TRIỂN KHAI BẰNG VIBE CODING VỚI COPILOT



BACKEND PYTHON FASTAPI

MỤC TIÊU

HIỆN THỰC HÓA NHANH YÊU CẦU NGHIỆP VỤ VÀ KIỂM CHỨNG THIẾT KẾ API.

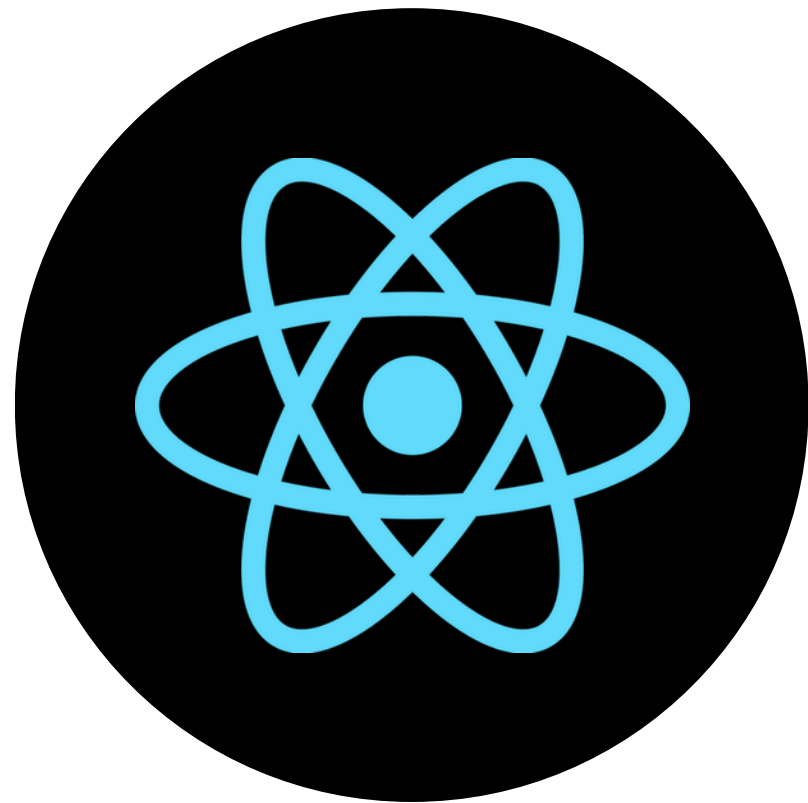
CÁCH TIẾP CẬN VIBE CODING

LẬP TRÌNH VIÊN MÔ TẢ MỤC TIÊU, RÀNG BUỘC QUA PROMPT. COPILOT HỖ TRỢ SINH CẤU TRÚC DỰ ÁN VÀ ENDPOINT TỪ TÀI LIỆU OPENAPI.

KẾT QUẢ

BACKEND CÓ CẤU TRÚC SẠCH, DỄ KIỂM TRA VÀ SẴN SÀNG DI TRÚ.

TRIỂN KHAI BẰNG VIBE CODING VỚI COPILOT



FRONTEND REACT

MỤC TIÊU

XÂY DỰNG GIAO DIỆN NGƯỜI DÙNG (UI) VÀ KIỂM CHỨNG LƯỒNG DỮ LIỆU QUA REST API.

ỨNG DỤNG COPILOT

PHÂN TÍCH THIẾT KẾ, TỰ ĐỘNG SINH CÁC REACT COMPONENTS VÀ LOGIC GỌI API.

KẾT QUẢ

TẠO RA MỘT FRONTEND CÓ CẤU TRÚC ĐỦ RÕ RÀNG ĐỂ PHỤC VỤ CHO QUÁ TRÌNH DI TRÚ SANG NỀN TẢNG KHÁC

TRIỂN KHAI BẢNG VIBE CODING VỚI COPILOT



BACKEND SPRING BOOT

MỤC TIÊU

ĐÁP ỨNG THAY ĐỔI VỀ CÔNG NGHỆ VÀ NGUỒN LỰC HỆ THỐNG.

CHIẾN LƯỢC AI-ASSISTED MIGRATION

CUNG CẤP TOÀN BỘ NGŨ CẢNH (MÃ NGUỒN CŨ + OPENAPI) CHO COPILOT AGENT MODE.

AI HỖ TRỢ ÁNH XẠ (MAPPING) LOGIC, MODEL DỮ LIỆU SANG SPRING BOOT.

KẾT QUẢ

SỬ DỤNG OPENAPI NHƯ "HỢP ĐỒNG BẤT BIẾN" ĐỂ HÀNH VI HỆ THỐNG KHÔNG THAY ĐỔI SAU KHI DI TRÚ.

TRIỂN KHAI BẰNG VIBE CODING VỚI COPILOT

Ụ

M C TIÊU

ĐỒNG BỘ HÓA CÔNG NGHỆ ỚI HỆ SINH THÁI .NET CỦA DOANH NGHIỆP.

ỨNG DỤNG COPILOT

COPILOT PHÂN TÍCH CẤU TRÚC COMPONENT REACT ĐỂ CHUYỂN ĐỔI TƯƠNG ĐƯƠNG SANG BLAZOR
ƯU TIÊN CÁC TÍNH NĂNG NATIVE BLAZOR, CHỈ DÙNG JSINTEROP KHI CẦN THIẾT.

KẾT QUẢ

GIẢM ĐÁNG KỂ THỜI GIAN HỌC FRAMEWORK MỚI, GI

Ữ VỮNG ĐỘ

ỔN ĐỊNH CỦA GIAO DIỆN.



FRONTEND BLAZOR

TRIỂN KHAI BẰNG VIBE CODING VỚI COPILOT



CONTAINERIZATION

MỤC TIÊU

SỬ DỤNG DOCKER ĐỂ ĐÓNG GÓI TOÀN BỘ HỆ THỐNG (SPRING BOOT & BLAZOR)

ỨNG DỤNG COPILOT

ÁP DỤNG MULTI-STAGE BUILD ĐỂ GIẢM DUNG LƯỢNG IMAGE.
DÙNG DOCKER COMPOSE ĐỂ THIẾT LẬP MẠNG NỘI BỘ VÀ BIẾN MÔI TRƯỜNG.

KẾT QUẢ

AI ĐÓNG VAI TRÒ TRỢ LÝ DEVOPS, CON NGƯỜI QUYẾT ĐỊNH VỀ KIẾN TRÚC TRIỂN KHAI VÀ BẢO MẬT.

ĐÁNH GIÁ VÀ KẾT LUẬN

ĐÁNH GIÁ HIỆU QUẢ CỦA VIBE CODING

- TĂNG TỐC ĐỘ PHÁT TRIỂN BAN ĐẦU
- HỖ TRỢ DI TRÚ CÔNG NGHỆ HIỆU QUẢ
- GIẢM KHỐI LƯỢNG CÔNG VIỆC LẶP LẠI
- TĂNG KHẢ NĂNG THỬ NGHIỆM VÀ THAY ĐỔI

HẠN CHẾ VÀ THÁCH THỨC

- PHỤ THUỘC VÀO CHẤT LƯỢNG PROMPT
- CẦN KIẾN THỨC NỀN TẢNG VỮNG
- KHÓ KIỂM SOÁT TRONG CÁC CHỨC NĂNG PHỨC TẠP
- CHƯA THAY THẾ HOÀN TOÀN LẬP TRÌNH VIÊN

ĐÁNH GIÁ VÀ KẾT LUẬN

BÀI HỌC RÚT RA

- Vibe Coding phù hợp cho khởi tạo hệ thống, sinh mã boilerplate và di trú công nghệ.
- Prompt cần rõ ràng, có cấu trúc và đầy đủ ngữ cảnh để tối ưu kết quả.
- Lập trình viên giữ vai trò kiểm soát kiến trúc và chất lượng hệ thống.
- OpenAPI giúp đảm bảo tính nhất quán của hệ thống sau quá trình di trú.
- Containerization hỗ trợ triển khai, tái lập và phát triển đa môi trường.

HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

- Tối ưu hiệu năng và khả năng mở rộng (Scalability) của hệ thống.
- Tăng cường bảo mật với OAuth2, Rate Limiting, Logging và Monitoring.
- Triển khai hệ thống trên nền tảng Cloud (AWS, Azure hoặc GCP).
- Tích hợp CI/CD Pipeline để tự động hóa build, test và deploy.
- Nghiên cứu ứng dụng AI trong kiểm thử tự động (AI-Assisted Testing)

**THANK YOU FOR
YOUR ATTENTION**